COMPUTER PRACTICAL- C 6 LAB 60 LECTURES

1. WRITE A PROGRAM (USING FORK() AND/OR EXEC() COMMANDS) WHERE PARENT AND CHILD EXECUTE:

A. SAME PROGRAM, SAME CODE.

ANSWER.

```
#!/bin/bash

# This script shows how parent and child (subshell) run the SAME CODE

echo "Parent PID = $$"     # $$ → PID of current script (parent)

(
   # Everything inside (...) is a SUBSHELL → behaves like CHILD process
   echo "Child PID = $$"   # $$ inside subshell gives CHILD PID
)
echo "Parent finished"     # Parent continues normally
```

B. SAME PROGRAM, DIFFERENT CODE.-

ANSWER.

```
#!/bin/bash

# Parent and child run DIFFERENT code sections

echo "Parent PID = $$"

(
   # CHILD process code
   echo "CHILD PROCESS"
   echo "Child PID = $$"
   echo "Child running DIFFERENT code"
```

```
) &        # & → run subshell in background like a child
echo "PARENT PROCESS"     # PARENT process code
echo "Parent running DIFFERENT code"
wait   # Ensures parent waits for child (optional)
```

C. BEFORE TERMINATING, THE PARENT WAITS FOR THE CHILD TO FINISH ITS TASK.

ANSWER.

```
#!/bin/bash

# Parent will wait until the child finishes using 'wait'

(
   echo "Child starting... PID = $$"
   sleep 3                    # Simulate child doing some task
   echo "Child finished."
) &

child_pid=$!                  # $! → PID of background child process

echo "Parent waiting for child (PID = $child_pid)..."

wait $child_pid               # Parent waits until child ends

echo "Parent ends AFTER child."     # Only runs after child completes
```

2. WRITE A PROGRAM TO REPORT BEHAVIOUR OF LINUX KERNEL INCLUDING KERNEL VERSION, CPU TYPE AND

MODEL. (CPU INFORMATION)

ANSWER.

```
#!/bin/bash

# This script reports Linux kernel and CPU information

echo "--------------------------------------"
echo "       LINUX KERNEL INFORMATION       "
echo "--------------------------------------"

# uname -r → prints kernel version
echo "Kernel Version:"
uname -r
echo ""

# uname -a → prints complete kernel details
echo "Full Kernel Details:"
uname -a
echo ""

echo "--------------------------------------"
echo "         CPU INFORMATION              "
echo "--------------------------------------"

# /proc/cpuinfo contains details about CPU
echo "CPU Model:"
grep -m 1 "model name" /proc/cpuinfo
echo ""

echo "CPU Vendor:"
grep -m 1 "vendor_id" /proc/cpuinfo
```

```
echo ""

echo "CPU MHz (speed):"
grep -m 1 "cpu MHz" /proc/cpuinfo
echo ""

echo "Total CPU Cores Available:"
grep -c "^processor" /proc/cpuinfo
echo ""
```

3. Write a program to report behaviour of Linux kernel including information on configured memory, amount of free and used memory. (memory information)

Answer.

```
#!/bin/bash

# This script displays configured, used, and free memory in Linux

echo "--------------------------------------"
echo "        MEMORY INFORMATION        "
echo "--------------------------------------"

# free -h → shows free, used, total RAM in human-readable format
echo "Memory Status (using free -h):"
free -h
echo ""

echo "Detailed Memory Information (from /proc/meminfo):"
# /proc/meminfo contains actual memory data line by line
```

```
echo "Total Memory:"
grep MemTotal /proc/meminfo      # Displays total physical RAM
echo ""

echo "Free Memory:"
grep MemFree /proc/meminfo       # Shows unused RAM
echo ""

echo "Available Memory:"
grep MemAvailable /proc/meminfo   # Shows memory that can be allocated
echo ""

echo "Buffers:"
grep Buffers /proc/meminfo       # Memory reserved for OS buffers
echo ""

echo "Cached Memory:"
grep ^Cached /proc/meminfo       # Memory used for cache
echo ""
```

4. Write a program to print file details including owner access permissions, file access time, where file name is given as argument.

Answer.

```
#!/bin/bash

# Check if the user provided a filename
if [ $# -eq 0 ]; then
   echo "Error: Please provide a filename"
   exit 1
```

```
fi

file=$1   # Store filename in variable

echo "---------- FILE INFORMATION ----------"

# ls -l shows permissions, owner, group, size, last modified time
echo "Basic Details (ls -l):"
ls -l $file
echo ""

# stat command gives detailed file information
echo "Access Time:"
stat -c %x $file     # %x → last access time
echo ""

echo "File Size (bytes):"
stat -c %s $file     # %s → file size
echo ""

echo "Permissions:"
stat -c %A $file     # %A → symbolic permissions
```

5. Write a program to copy files using system calls.

Answer.

```
#!/bin/bash

# Check if two file names are given
if [ $# -ne 2 ]; then
   echo "Usage: ./copy.sh source_file destination_file"
```

```
    exit 1
fi

src=$1
dest=$2

# cp command copies file (replicates read/write system calls)
cp $src $dest

echo "File copied successfully from $src to $dest"
```

6. Write program to implement FCFS scheduling algorithm.

Answer.

```
#!/bin/bash

# First Come First Serve Scheduling Algorithm

echo -n "Enter number of processes: "
read n

# Input burst time for each process
for ((i=1; i<=n; i++))
do
    echo -n "Burst Time of P$i: "
    read bt[$i]
done

# Waiting time of first process = 0
wt[1]=0
```

```
# Waiting time for rest of processes
for ((i=2; i<=n; i++))
do
   wt[$i]=$((wt[$i-1] + bt[$i-1]))
done

echo "Process   BT   WT   TAT"

# Turnaround time = BT + WT
for ((i=1; i<=n; i++))
do
   tat[$i]=$((wt[$i] + bt[$i]))
   echo "P$i      ${bt[$i]}    ${wt[$i]}    ${tat[$i]}"
done
```

7. Write program to implement Round Robin scheduling algorithm.

Answer.

```
#!/bin/bash

echo -n "Enter number of processes: "
read n

echo -n "Enter Time Quantum: "
read tq

# Input burst times
for ((i=1; i<=n; i++))
```

```
do
    echo -n "Burst Time of P$i: "
    read bt[$i]
    rt[$i]=${bt[$i]}    # rt[] = remaining time
done

time=0
flag=1

# Continue until all processes are completed
while [ $flag -eq 1 ]
do
    flag=0
    for ((i=1; i<=n; i++))
    do
        if [ ${rt[$i]} -gt 0 ]; then
            flag=1

            if [ ${rt[$i]} -gt $tq ]; then
                time=$((time + tq))
                rt[$i]=$((rt[$i] - tq))
            else
                time=$((time + rt[$i]))
                tat[$i]=$time
                rt[$i]=0
            fi
        fi
    done
done
```

```
# Calculate waiting time
for ((i=1; i<=n; i++))
do
   wt[$i]=$((tat[$i] - bt[$i]))
done

echo "Process BT WT TAT"
for ((i=1; i<=n; i++))
do
   echo "P$i ${bt[$i]} ${wt[$i]} ${tat[$i]}"
done
```

8. Write program to implement SJF scheduling algorithm.

Answer.

```
#!/bin/bash

echo -n "Enter number of processes: "
read n

# Input burst times
for ((i=1; i<=n; i++))
do
   echo -n "Burst Time of P$i: "
   read bt[$i]
   p[$i]=$i    # store process number
done

# Sort burst times (ascending)
```

```
for ((i=1; i<=n; i++))
do
   for ((j=i+1; j<=n; j++))
   do
      if [ ${bt[$i]} -gt ${bt[$j]} ]; then
         temp=${bt[$i]}
         bt[$i]=${bt[$j]}
         bt[$j]=$temp

         temp2=${p[$i]}
         p[$i]=${p[$j]}
         p[$j]=$temp2
      fi
   done
done

wt[1]=0

# Calculate waiting time
for ((i=2; i<=n; i++))
do
   wt[$i]=$((wt[$i-1] + bt[$i-1]))
done

echo "Process  BT  WT  TAT"

# TAT = BT + WT
for ((i=1; i<=n; i++))
do
```

```
    tat[$i]=$((wt[$i] + bt[$i]))
    echo "P${p[$i]}  ${bt[$i]}  ${wt[$i]}  ${tat[$i]}"
done
```

9. Write program to implement non-preemptive priority based scheduling algorithm.

Answer.

```
#!/bin/bash

echo -n "Enter number of processes: "
read n

# Take burst time & priority input
for ((i=1; i<=n; i++))
do
    echo -n "Burst time of P$i: "
    read bt[$i]
    echo -n "Priority of P$i (lower = higher priority): "
    read pr[$i]
    p[$i]=$i
done

# Sort according to priority
for ((i=1; i<=n; i++))
do
    for ((j=i+1; j<=n; j++))
    do
        if [ ${pr[$i]} -gt ${pr[$j]} ]; then
            temp=${pr[$i]}; pr[$i]=${pr[$j]}; pr[$j]=$temp
```

```
            temp2=${bt[$i]}; bt[$i]=${bt[$j]}; bt[$j]=$temp2
            temp3=${p[$i]}; p[$i]=${p[$j]}; p[$j]=$temp3
        fi
    done
done

wt[1]=0

# Calculate waiting time
for ((i=2; i<=n; i++))
do
    wt[$i]=$((wt[$i-1] + bt[$i-1]))
done

echo "Process  BT  WT  TAT  Priority"

# TAT = BT + WT
for ((i=1; i<=n; i++))
do
    tat[$i]=$((bt[$i] + wt[$i]))
    echo "P${p[$i]}   ${bt[$i]}   ${wt[$i]}   ${tat[$i]}   ${pr[$i]}"
done
```

10. Write program to implement preemptive priority based scheduling algorithm.

Answer.

```
#!/bin/bash

echo -n "Enter number of processes: "
```

```
read n

# Input burst time, priority, and initialize remaining time
for ((i=1; i<=n; i++))
do
   echo -n "Burst Time of P$i: "
   read bt[$i]
   echo -n "Priority of P$i: "
   read pr[$i]
   rt[$i]=${bt[$i]}
done

time=0
count=$n

# Continue until all processes complete
while [ $count -gt 0 ]
do
   min_priority=9999
   selected=0

   # Select highest priority process (lowest pr value)
   for ((i=1; i<=n; i++))
   do
      if [ ${rt[$i]} -gt 0 ] && [ ${pr[$i]} -lt $min_priority ]; then
         min_priority=${pr[$i]}
         selected=$i
      fi
   done
```

```
    # Execute 1 unit of CPU time
    rt[$selected]=$((rt[$selected] - 1))
    time=$((time + 1))

    # If finished
    if [ ${rt[$selected]} -eq 0 ]; then
        tat[$selected]=$time
        wt[$selected]=$((tat[$selected] - bt[$selected]))
        count=$((count - 1))
    fi
done

echo "Process BT WT TAT Priority"
for ((i=1; i<=n; i++))
do
    echo "P$i  ${bt[$i]}  ${wt[$i]}  ${tat[$i]}  ${pr[$i]}"
done
```

11. Write program to implement SRJF scheduling algorithm.

Answer.

```
#!/bin/bash

echo -n "Enter number of processes: "
read n

# Input burst time & initialize remaining time
for ((i=1; i<=n; i++))
do
```

```
    echo -n "Burst Time P$i: "
    read bt[$i]
    rt[$i]=${bt[$i]}
done

time=0
count=$n

# Continue until all processes finish
while [ $count -gt 0 ]
do
    min_bt=9999
    selected=0

    # Choose process with smallest remaining time
    for ((i=1; i<=n; i++))
    do
        if [ ${rt[$i]} -gt 0 ] && [ ${rt[$i]} -lt $min_bt ]; then
            min_bt=${rt[$i]}
            selected=$i
        fi
    done

    # Execute 1 unit
    rt[$selected]=$((rt[$selected] - 1))
    time=$((time + 1))

    if [ ${rt[$selected]} -eq 0 ]; then
        tat[$selected]=$time
```

```
        wt[$selected]=$((tat[$selected] - bt[$selected]))
        count=$((count - 1))
    fi
done

echo "Process BT WT TAT"
for ((i=1; i<=n; i++))
do
    echo "P$i ${bt[$i]} ${wt[$i]} ${tat[$i]}"
done
```

12. Write program to calculate sum of n numbers using thread library.

Answer.

```
#!/bin/bash

# Shell does not support threads, so we simulate logic

echo -n "Enter n: "
read n

sum=0

# Summation using loop
for ((i=1; i<=n; i++))
do
    sum=$((sum + i))
done
```

```
echo "Sum of first $n numbers = $sum"
```

13. Write a program to implement first-fit, best-fit and worst-fit allocation strategies.

Answer.

```
#!/bin/bash

echo -n "Enter number of memory blocks: "
read nb

echo "Enter block sizes:"
for ((i=1; i<=nb; i++))
do
   read block[$i]
done

echo -n "Enter number of processes: "
read np

echo "Enter process sizes:"
for ((i=1; i<=np; i++))
do
   read process[$i]
done

echo "----- FIRST FIT -----"

for ((i=1; i<=np; i++))
do
```

```
    allocated=0
    for ((j=1; j<=nb; j++))
    do
        if [ ${block[$j]} -ge ${process[$i]} ]; then
            echo "Process $i allocated to Block $j"
            block[$j]=$((block[$j] - process[$i])) # Reduce block size
            allocated=1
            break
        fi
    done

    if [ $allocated -eq 0 ]; then
        echo "Process $i could NOT be allocated"
    fi
done
```